STRING CONUNDRUM

Hlomla Kuhlane
Department of Computer Science
University of Cape Town
Cape Town, South Africa
KHLHLO002@myuct.ac.za

Luvuyo Matomela
Department of Computer Science
University of Cape Town
Cape Town, South Africa
MTMLUV001@myuct.ac.za

ABSTRACT

Strings are among the most frequently assessed and most poorly understood concepts in introductory programming. Despite decades of research into novice programmer difficulties, no widely adopted educational tool exists that specifically targets string misconceptions. This proposal presents String Conundrum, a web-based interactive educational tool designed to help novice programmers develop genuine understanding of string concepts. The tool combines a visual string representation system, a code input and transformation pipeline, a structured concept progression, and a misconception-targeted feedback engine. It will be evaluated through usability testing and a comparative learning study with first-year Python students at the University of Cape Town.

1 INTRODUCTION

Strings are fundamental data types present in almost every programming language. They are essential for displaying information to users, processing input, creating error messages, and building text-based applications [7, 8]. Despite their central role, strings are not a simple concept for novice programmers to master. Students must understand how text is stored as an indexed sequence of characters in memory, how individual characters and substrings are accessed, how strings behave differently from other data types during comparison and concatenation, and why strings cannot be modified in place [9]. This combination of syntactic and conceptual demands makes strings among the most difficult introductory programming topics.

Watson and Li [1] found that approximately one in three students fails or withdraws from introductory programming courses globally, a rate unchanged for over fifty years. Altadmri and Brown [2] analysed over 37 million compilation events from more than 250,000 students and found that string comparison errors alone affected nearly 30,000 distinct students with a median fix time exceeding 1000 seconds. Izu and Weerasinghe [3] found that only 49% of students successfully completed a string manipulation task after a full semester of instruction, and only 6% used string library functions they had been explicitly taught one week prior. These findings suggest that current approaches to teaching strings do not produce genuine or transferable understanding.

no widely adopted tool currently exists that specifically targets string misconceptions. General-purpose visualisation tools such as Online Python Tutor [4] and misconception-targeting systems such as the SMoL Tutor [5] address broader programming concepts but were not designed around the unique challenges strings present. Alasmari et al. [6] confirmed through a systematic review of seven online coding platforms that none addressed string-specific difficulties. The gap remains unfilled.

This project addresses that gap. We present String Conundrum, a web-based interactive educational tool designed to help novice Python programmers understand string concepts. At UCT, student marks consistently drop when string concepts are introduced, confirming that the difficulties documented in the international literature are present in the local context. The tool presents string concepts in a structured progression, visualises strings as indexed character sequences, and delivers immediate misconception-targeted feedback when students answer incorrectly. The goal is to produce the genuine conceptual understanding that existing instruction and tools have consistently failed to provide.

2 RELATED WORK

Novice programmers consistently struggle with string concepts for reasons that theory and empirical research both explain. Qian and Lehman [9] identify inaccurate mental models as one of the most persistent sources of difficulty in introductory programming, noting that conceptual errors are harder to address than syntax errors because they produce no immediate visible feedback. Sorva [11] extends this through the notional machine framework, arguing that students struggle when their internal model of how a program executes does not match actual language behaviour. Applied to strings, a student who does not understand that strings are immutable will consistently fail at operations that depend on that property regardless of how many times they encounter the correct syntax. Boustedt et al. [10] argue that certain concepts function as threshold concepts ideas that, once genuinely understood, transform a student's broader understanding. String immutability plausibly meets this criterion.

The empirical evidence confirms that string difficulties are severe and persistent. Altadmri and Brown [2] found string comparison errors among the hardest to fix in their dataset of 37 million compilation events. Izu and Weerasinghe [3] found that only 6% of students applied string functions they had been explicitly taught one week prior, demonstrating that traditional instruction does not produce transferable understanding.

Existing educational tools do not address these difficulties. Guo [4] presents Online Python Tutor, a widely used program

visualisation tool that makes string behaviour visible but provides no guidance targeted at string-specific misconceptions. VanLehn [13] demonstrated that intelligent tutoring systems produce learning gains nearly equivalent to one-on-one human tutoring, and Corbett and Anderson [12] established knowledge tracing as an effective mechanism for tracking concept-level mastery within such systems. Neither approach has been applied to string concepts specifically. Alasmari et al. [6] confirmed through a systematic review of seven online coding platforms that none addressed string-specific difficulties. String Conundrum addresses this gap by combining structured concept progression, visual string representation, and misconception-targeted feedback within a single tool built specifically around the challenges strings present.

3 PROBLEM STATEMENT & AIMS

3.1 Problem Statement

Novice Python programmers consistently fail to develop genuine understanding of string concepts despite receiving direct instruction on them. String-related errors are among the most frequent and persistent in introductory programming, yet no educational tool exists that specifically targets the conceptual difficulties strings present. Existing tools either visualise general program execution without addressing string-specific misconceptions or target broader programming concepts without the string-focused guidance that novice learners need. The result is that students encounter string concepts repeatedly across their introductory course without ever developing the correct mental model needed to apply them reliably. At UCT, this problem is visible in the consistent drop in student marks for practical assessments and assignments when string concepts are introduced. This project addresses that gap by designing and developing a dedicated web-based educational tool that targets string misconceptions directly, presents string concepts in a structured progression grounded in theory, and delivers misconception-targeted feedback that goes beyond simply showing students the correct answer.

3.2 Limitations of Existing Tools

The educational tools currently available to novice programmers do not address string-specific difficulties. General-purpose visualisers such as Online Python Tutor [4] make string behaviour visible as part of a broader view of program execution but provide no guidance targeted at string misconceptions. A student using Online Python Tutor to step through a string operation must notice on their own that a new object was created, reason about why it matters, and connect it to their existing mental model without any support from the tool. Misconception-targeting systems such as the SMoL Tutor [5] address scope, mutation, and aliasing but were not designed around the specific conceptual challenges strings present. Alasmari et al. [6] confirmed through a systematic review of seven online coding platforms that none addressed string-specific difficulties and that all were missing key features important for novice learners. Static analysis tools can detect string errors but cannot explain the underlying conceptual misunderstanding to the student [16].

3.3 Target Users & Needs

String Conundrum targets first-year university students enrolled in introductory Python programming courses, particularly those encountering string concepts for the first time. At UCT, student performance data shows a consistent drop in marks for practical assessments and assignments when string concepts are introduced, confirming that the difficulties documented in the international literature are present in the local context. At this stage, students often struggle to form accurate mental models of how strings behave, leading to persistent misconceptions that affect their ability to correctly apply string operations.

Prior research identifies a consistent set of needs among these students. Students need a correct mental model of strings as indexed sequences of characters before they can reason correctly about indexing, slicing, immutability, and comparison [3, 11]. They need immediate feedback that addresses the specific misconception behind a wrong answer rather than simply showing the correct one [11, 5]. They need string concepts presented before syntax so that conceptual understanding precedes syntactic demands [14], and a low cognitive load environment that directs mental effort toward string behaviour rather than interface navigation [15].

3.4 Project Aims

This project aims to design and develop String Conundrum, a web-based interactive educational tool for novice Python programmers learning string concepts. Specifically, the project aims to:

(1) Develop a visual string representation system that displays strings as indexed sequences of characters, where relevant characters highlight dynamically as students observe or perform string operations.

(2) Develop a code input and transformation pipeline that accepts Python string code as input, parses and evaluates it, and transforms the result into the visual representation displayed to the student.

(3) Develop a structured exercise system that presents string concepts in a principled progression from fundamental to complex, using multiple choice and fill-in-the-blank questions with real Python code.

(4) Develop a misconception-targeted feedback system that identifies the specific misconception underlying an incorrect answer and provides a clear, plain-language explanation that directly addresses it.

(5) The fifth aim is to evaluate String Conundrum through usability testing and a comparative learning study with first-year Python students at UCT, measuring pre and post-test knowledge gain.

4 PROCEDURES & METHODS

4.1 System Architecture

String Conundrum will use a Model-View-Controller architecture, with Firebase Firestore serving as the Model to store user progress, scores, and session data, while Firebase Authentication manages user identity. The View will be a React web application that runs in the browser and requires no installation, and the Controller will be a Flask backend that exposes a REST API and handles all communication between the View and Model.

Each layer will be developed and tested independently. Deployment will be automated using GitHub Actions. This approach ensures separation of concerns, enabling independent development, testing, and scalability.

The system architecture is illustrated in Figure 1 (see Appendix D).

4.2 The Visualisation View

The Visualisation View will display strings as sequences of character cells. Each cell will show a character and its index. Operations such as indexing, slicing, and concatenation will highlight relevant characters. When a new string is created, a second object will be shown below the original. A labelled arrow will indicate the transformation. This will make string immutability visible. The frontend will not compute state. It will only render backend data.

4.3 Code Space and Learning View

4.3.1 The Code Space View will provide a guided mini-IDE. It will allow students to experiment with string expressions. The interface will include a code editor and output panel. Students will use pre-defined templates and prompts. Submitted code will be executed in a sandbox. The backend will return output and string state data. The Visualisation View will update using this data. This view will support exploration and deeper understanding.

4.3.2 The Learning View will present structured exercises. It will include multiple-choice and fill-in-the-blank questions. Each task will use Python string operations. Students will answer questions based on provided code. Answers will be sent to the backend for evaluation. The system will detect correctness and identify misconceptions. Feedback will be returned in plain language. This view will guide students through concept progression.

4.4 Architecture and Frontend Integration

The React frontend and Flask backend will communicate exclusively through a REST API. The API contract will be defined and frozen before development begins to prevent integration failures. All endpoints will return JSON. The frontend will not compute string state or evaluates answers; it will only render data returned by the backend. Authentication tokens issued by Firebase Authentication will be forwarded with every API request in the Authorization header, ensuring that progress records are tied to authenticated identities and cannot be spoofed.

A more detailed view of the component interactions is shown in Figure 2 (see Appendix D).

4.5 Infrastructure: Persistence, Deployment, and DevOps

Student progress will be stored in Firebase Firestore as a document per user, keyed by the Firebase Authentication UID. Each document will record, for each concept module, whether the student has completed the module, their most recent score, and the timestamp of their last session. The React frontend will be deployed to Firebase Hosting. The Flask backend will be deployed to the UCT Computer Science Department server, with Render (free tier) provisioned as a failover.

4.6 Development Platform and Frameworks

The frontend will be built with React using functional components and the useState/useEffect hooks pattern. No external UI component library will be used; all components will be custom-built to keep the interface cognitively lightweight, in line with Kolling's design principle [14]. Styling will be implemented in plain CSS modules. The backend will be built with Flask running on Python. The code execution sandbox will use Python's subprocess module with a timeout wrapper and a restricted globals dictionary that excludes all standard library imports. Version control will be managed in a shared GitHub repository with a branch-based workflow; feature branches will be reviewed via pull request and merged to main only when associated tests pass.

4.7 Development Phases

4.7.1 Phase 1 Infrastructure Setup

Phase 1 will establish the project infrastructure and deliver the minimum viable architecture. Tasks will include initialising the GitHub repository with CI configuration; setting up the Firebase project (Authentication, Firestore, Hosting); scaffolding the React application with routing; scaffolding the Flask application with the four REST endpoints returning stub responses; and defining and documenting the API contract. Phase 1 will be complete when the React frontend will be able to reach the Flask backend, receive a stub response, and display it on screen, confirming that the full communication path functions end-to-end.

4.7.2 Phase 2 Visualiser and Code Execution Pipeline

Phase 2 will deliver the two primary features: the string visualiser and the code execution pipeline. Backend tasks will include implementing the code execution sandbox and the expression parser that produces the JSON state representation. Frontend tasks will include implementing the Visualisation View in full, including dynamic character highlighting and new-object rendering for immutability operations. Phase 2 will also deliver the first complete concept module Introduction to Strings including exercises, multiple-choice questions, and misconception-targeted feedback for each incorrect answer option.

4.7.3 Phase 3 Concept modules

Phase 3 will complete the remaining concept modules: String Indexing and Zero-Based Counting; String Slicing and Exclusive End Bounds; String Immutability; String Comparison; Looping Through Strings; String Length and Built-in Methods; and String Formatting. Each module will follow the same structure as the Introduction module. The knowledge-tracing progression logic will be implemented, gating later modules behind a minimum score on earlier ones. The leaderboard and progress dashboard will be delivered in this phase if time permits.

4.7.4 Phase 4: Integration, Evaluation, and Completion

Phase 4 will integrate all components and resolve cross-layer issues identified during internal testing. The leaderboard and progress dashboard will be completed. Functional, usability, comparative learning, and performance testing described in Sections 4.9.1 to 4.9.4 will be conducted. Final refinements will be made before the paper submission deadline.

4.8 Expected Technical Challenges

The most significant technical challenge will be the safe execution of student-submitted Python code. The subprocess sandbox with a timeout wrapper and a restricted globals dictionary will address injection, resource exhaustion, and filesystem access; it will be validated through a dedicated security session before deployment.

A second challenge will be rendering all edge cases like unicode characters, negative indices, and out-of-range slices correctly in the Visualisation View. Any discrepancy would reinforce the faulty notional machine students will already hold [11]. A comprehensive unit test suite covering every operation type will address this.

A third challenge will be maintaining integration across parallel development streams. The frozen API specification agreed in Phase 1 and weekly synchronisation meetings will address this.

A fourth challenge will be concurrent reliability. Load testing with the Locust framework simulating at least twenty concurrent users before the evaluation session will address this.

4.9 System Evaluation

4.9.1 Functional Testing

Each REST API endpoint will be tested with unit tests covering valid inputs, invalid inputs, and boundary conditions. The sandbox will be tested for security boundaries: import attempts, infinite loops, and memory-intensive operations will have to be intercepted correctly. Visualiser snapshot tests will verify character-cell layout for a fixed set of operations. All tests will run automatically on every commit via GitHub Actions.

4.9.2 Usability Testing

Usability testing will be conducted in one of the UCT Computer Science computer labs, where 15 to 20 first-year Python students will be invited to attend a scheduled session. Each participant will be seated at a lab with String Conundrum Educational tool loaded in the browser and asked to work through the Introduction to Strings and String Immutability modules independently, without assistance from the research team. An observer will sit nearby to record points of confusion, navigation difficulties, and problematic interface elements as they occur. Immediately after completing the modules, each participant will fill in a usability survey on the same workstation. The survey will use the System Usability Scale (SUS), a standardised ten-item questionnaire that produces a quantitative score benchmarked against established thresholds, supplemented by short free-response questions asking participants how they felt about the application's ease of use, clarity of feedback, and overall experience. Aggregated SUS scores and observer notes will then be analysed to identify usability issues for remediation.

4.9.3 Comparative Learning Evaluation

Participants will complete a pre-test covering string indexing, slicing, immutability, and comparison. The experimental group will use String Conundrum; the control group will receive standard course materials. Both will complete the same post-test. Learning gain will be compared using an independent-samples t-test with Cohen's d effect size.

4.9.4 Performance and Stability Testing

Load testing using the Locust framework will verify that the backend will handle at least twenty concurrent requests within a two-second response time. Any regression will be resolved before participants are recruited. CPU and memory usage will be monitored throughout the evaluation session.

5 ETHICAL, PROFESSIONAL & LEGAL ISSUES

String Conundrum involves user testing with first-year Python programming students at UCT. Ethical clearance from the Faculty of Science Research Ethics Committee will be obtained before any participant is recruited or any testing begins. All participants will be briefed on the purpose of the study and the nature of the tasks they will be asked to complete. Participation is entirely voluntary and participants may withdraw at any stage without consequence. Prior to taking part, each participant will read and sign an informed consent form. The application does not store any personally identifying information. Progress and answer data collected during testing will be anonymised and used solely for the purpose of evaluating the system.

The intellectual property produced by this project is owned by the University of Cape Town in line with university policy on student project outputs [17]. The source code will be made publicly available under an open-source licence. To formalise this, the team will submit the relevant licence agreement to the UCT Research Contracts and Intellectual Property Services office for review and approval.

6 ANTICIPATED OUTCOMES

6.1 Functional System

The primary outcome of this project is a functional, deployed web application that novice Python programmers can access through

any browser without installation. The system is intended to comprise a visual string representation component with dynamic highlighting, a code input and transformation pipeline, a structured exercise system covering string concept modules in a principled progression, and a misconception-targeted feedback engine. Progress tracking, scoring, and a leaderboard are intended to be accessible to all users throughout their session. If time permits, an isiXhosa language transpiler will be implemented as an additional optional feature, accessible via a language toggle, allowing students to interact with the tool in isiXhosa in response to UCT's commitment to multilingual student engagement.

6.2 Expected Impact

String Conundrum aims to fill a gap that existing computing education research and tooling have left unaddressed. No dedicated tool for teaching string concepts currently exists, and research has consistently shown that traditional instruction does not produce genuine or transferable understanding of strings [3, 8]. At UCT, where student marks consistently drop when string concepts are introduced, the tool is expected to improve student performance in practical assessments, tests, and examinations that assess string understanding. Students who use the tool are expected to develop a correct mental model of string behaviour, leading to greater confidence, self-efficacy, and engagement with string-related tasks in their coursework. The tool is intended to be made openly available so that other institutions teaching introductory Python can adopt and build on it.

6.3 Success Metrics

The project will be considered successful if the following outcomes are achieved:

- The system is fully deployed and accessible through a browser with all string concept modules functional.
- Usability testing demonstrates that first-year Python students can navigate and use the tool without assistance, with a SUS score above the established benchmark threshold.
- Participants are able to complete each concept module end-to-end without abandoning the task, with a task completion rate of at least 80 percent across the usability testing cohort.
- The misconception-targeted feedback engine returns the correct misconception-specific response in at least 90 percent of cases where a wrong answer maps to a known misconception, verified against a curated test set of misconception-triggering inputs.
- Backend response time for code execution and feedback generation remains under two seconds for at least 95 percent of requests during a load test simulating twenty concurrent users.
- The misconception-targeted feedback system correctly identifies and addresses the anticipated misconception for each wrong answer across all concept modules.

7 PROJECT PLAN

7.1 Risk and Mitigation

This section identifies the key risks that could affect the successful completion of the project. A complete risk matrix covering all identified risks including cause, trigger, impact, probability, mitigation strategy, contingency plan, and monitoring approach is provided in Appendix A.

7.2 Timeline

The project is organised into four development phases (described in Section 4.7) and an evaluation phase, followed by report writing and submission. The full Gantt chart illustrating the schedule across the academic year is provided in Appendix B.

7.3 Resources Required

Software resources include React, Flask, Firebase Firestore, Firebase Authentication, Firebase Hosting, and Git. We will investigate the suitability of each framework and select alternatives where necessary. Infrastructure resources include either a Render deployment environment or a server provisioned by the UCT Computer Science Department. Human resources include first-year Python students at UCT recruited as participants for usability testing and the comparative learning evaluation, with a target of at least 15 participants per evaluation activity. No specialised hardware or commercial software licences are required; all tools and frameworks are freely available.

7.4 Deliverables

The concrete outputs the project will produce, with their associated due dates, are listed in Appendix C.

7.5 Milestones

The key technical achievements that mark progress toward project completion are listed in Appendix C.

7.6 Work Allocation

Hlomla Kuhlane is responsible for the frontend and user-facing components of String Conundrum. His individual contribution encompasses designing the UI mock-ups and user flows, implementing the visual string representation component including indexed character cells and dynamic highlighting, building the complete React user interface covering inputs, navigation, and module layout, handling all user interactions such as clicks, inputs, and module progression, displaying feedback and error messages returned by the backend, rendering the progress dashboard, scoring, and leaderboard views, and integrating the Firebase Authentication UI including login, signup, and logout flows.

Luvuyo Matomela is responsible for the backend, data pipeline, and evaluation logic. His individual contribution encompasses implementing the sandboxed Python string execution engine, generating the structured JSON visualisation data consumed by the frontend, building the misconception detection system that

maps incorrect answers to specific conceptual errors, producing the targeted feedback explanations delivered to students, managing all Firestore database interactions including reads and writes, computing scores and leaderboard rankings, designing and implementing the REST API endpoints, and validating Firebase Authentication tokens on all protected routes.

Both team members contribute jointly to the API contract definition, system architecture decisions, integration testing, Firebase project setup, user role design, string concept module design, exercise creation and misconception identification, usability and learning evaluation, report writing and review, the ethics application, and weekly synchronisation meetings.

8 AI USAGE STATEMENT

During the preparation of this proposal, the team used two generative AI tools, OpenAI's ChatGPT and Google's Gemini, solely for feedback on our own writing. Specifically, the tools were used to (i) review the structure and organisation of the proposal and suggest where sections could be reordered or tightened, (ii) identify weaknesses in clarity and argumentation in passages we had already drafted, and (iii) proofread for grammar, punctuation, and consistency of tense and terminology. No text was generated by either tool and copied into this document. All writing in this proposal is our own. We critically evaluated each suggestion before deciding whether to act on it, and we accepted only those suggestions that we judged to improve the clarity or correctness of our own ideas. No references, citations, or factual claims in this proposal were sourced from a generative AI tool.

REFERENCES

[1] Christopher Watson and Frederick W.B. Li. 2014. Failure Rates in Introductory Programming Revisited. In Proc. ITiCSE 2014. ACM, New York, 39–44.

[2] Amjad Altadmri and Neil C.C. Brown. 2015. 37 Million Compilations: Investigating Novice Programming Mistakes in Large-Scale Student Data. In Proc. SIGCSE 2015. ACM, New York, 522–527.

[3] Cruz Izu and Amali Weerasinghe. 2020. Assessing CS1 Design Skills with a String Manipulation Task. In Proc. ITiCSE 2020. ACM, New York, 432–438.

[4] Philip J. Guo. 2013. Online Python Tutor: Embeddable Web-Based Program Visualization for CS Education. In Proc. SIGCSE 2013. ACM, New York, 579–584.

[5] Kuang-Chen Lu and Shriram Krishnamurthi. 2024. Identifying and Correcting Programming Language Behavior Misconceptions. Proc. ACM Program. Lang. 8, OOPSLA1, Article 106.

[6] Ohud Alasmari, Jeremy Singer, and Mireilla Bikanga Ada. 2023. Do Current Online Coding Tutorial Systems Address Novice Programmer Difficulties? In Proc. ICETC 2023. ACM, New York, 242–248.

[7] Andrew Luxton-Reilly and Andrew Petersen. 2017. The Compound Nature of Novice Programming Assessments. In Proc. ACE 2017. ACM, New York, 26–35.

[8] Michael C. Hughes, Matthew C. Jadud, and Ma. Mercedes T. Rodrigo. 2010. String Formatting Considered Harmful for Novice Programmers. Computer Science Education 20, 3, 201–228.

[9] Yizhou Qian and James Lehman. 2017. Students' Misconceptions and Other Difficulties in Introductory Programming: A Literature Review. ACM TOCE 18, 1, Article 1.

[10] Jonas Boustedt, Anna Eckerdal, Robert McCartney, Jan Erik Moström, Mark Ratcliffe, Kate Sanders, and Carol Zander. 2007. Threshold Concepts in Computer Science: Do They Exist and Are They Useful? In Proc. SIGCSE 2007. ACM, New York, 504–508.

[11] Juha Sorva. 2013. Notional Machines and Introductory Programming Education. ACM TOCE 13, 2, Article 8.

[12] Albert T. Corbett and John R. Anderson. 1994. Knowledge Tracing: Modelling the Acquisition of Procedural Knowledge. User Modeling and User-Adapted Interaction 4, 4, 253–278.

[13] Kurt VanLehn. 2011. The Relative Effectiveness of Human Tutoring, Intelligent Tutoring Systems, and Other Tutoring Systems. Educational Psychologist 46, 3, 197–221.

[14] Michael Kolling. 2010. The Greenfoot Programming Environment. ACM TOCE 10, 4, Article 14.

[15] Vesa Vainio and Jorma Sajaniemi. 2007. Factors in Novice Programmers' Poor Tracing Skills. In Proc. ITiCSE 2007. ACM, New York, 236–240.

[16] Tim Blok and Ansgar Fehnker. 2017. Automated Program Analysis for Novice Programmers. In Proc. HEAD'17. Universitat Politècnica de València, 1–8.

[17] University of Cape Town. 2011. Intellectual Property Policy. University of Cape Town, Cape Town, South Africa. https://www.uct.ac.za/sites/default/files/image_tool/images/328/about/policies/intellect_property_policyrev2011.pdf

APPENDIX A: RISK MATRIX

Risk	Category	Cause	Trigger	Impact	Probability	Mitigation	Contingency	Monitoring
Technical – Backend Execution	Backend Functionality	Flask sandbox must execute arbitrary student-submitted Python code safely	Student submits malicious or infinite-loop code	Server crash, resource exhaustion, or security breach	Medium	Wrap execution in subprocess with CPU/memory limits; disallow os/sys/subprocess imports	Fall back to pre-loaded example outputs; disable free-form input until patched	Monitor CPU/memory; log all execution errors; review weekly
Technical Visualiser Accuracy	User Experience	Visualiser must correctly reflect Python's memory model for all operations	Edge case (Unicode, negative indices, multi-byte slices) not handled correctly	Students receive misleading feedback, reinforcing misconceptions	Medium	Unit tests for every visualiser operation; use Python AST module not regex	Disable affected feature; display plain-text output with note until fixed	Run test suite on every commit via CI; track bug reports from user testing
Technical – Deployment Availability	Resource	Flask backend requires a server (UCT CS or Render)	UCT server denied or Render free tier goes down during evaluation	Tool inaccessible to participants during evaluation window	Low	Provision both UCT server and Render in parallel as failover	Switch DNS to backup within the hour; notify participants via LMS	Uptime monitoring (UptimeRobot) with email alerts for downtime > 5 min
Evaluation – Ethics Clearance Delay	Schedule	Ethics clearance from Faculty REC must be obtained before any user testing	Application takes longer than expected to process	User testing cannot begin on schedule, compressing evaluation phase	Medium	Submit ethics application six weeks before planned testing; prepare all instruments in advance	Replace participant testing with think-aloud sessions with non-UCT-CS volunteers	Track application weekly; escalate to supervisor if no response within three weeks
Project Integration Complexity	Functional complexity	React, Flask, and Firebase integrated across two parallel development streams	API contract changes by one member break the other's implementation	Integration delayed; final system incomplete at hand-in	Medium	Freeze REST API contract before development; schema changes require both members to sign off	Prioritise core features (visualiser + exercises) over secondary features	Weekly sync to review integration status and flag API deviations immediately
Feature creep	Scope Management	Underestimation of implementation and translation effort	Non-essential features added beyond original scope	Core features incomplete by deadline	High	Strictly follow the five project aims; language support as optional extension only	Descope non-essential features; abandon language support if core features are at risk	Review feature list at the start of each development phase
Students unable to use tool without assistance	User Experience	Interface design not sufficiently intuitive for target users	Students abandon tool during usability testing	Tool fails usability testing criterion	Medium	Apply user-centred design principles; iterative testing throughout development	Redesign problematic components identified during usability testing	Monitor task completion and abandonment rates throughout development
Feedback messages addressing wrong misconception	User Experience	Misconception model incorrectly mapping wrong answers to misconceptions	Students receive irrelevant feedback for incorrect answers	Tool fails to correct misconceptions	Medium	Manually review and validate each misconception mapping before deployment	Update misconception mappings based on usability testing observations	Review feedback accuracy during functional testing

Merge conflicts breaking functionality	Team Coordination	Overlapping work on shared components	Conflicting changes merged into main branch	Features break in deployed system	Medium	Use feature branch workflow and review all pull requests before merging	Revert to last stable branch and resolve conflicts manually	Review GitHub commit history weekly
API contract changes breaking integration	Team Coordination	One member changes endpoint format without notifying the other	Frontend or backend stops functioning correctly	Integration failures requiring rework	Medium	Document all API changes and notify the other member before implementing	Revert API change and agree on updated contract	Review API documentation after every change
Academic workload causing delays	Scheduling	Workload from other courses competing with project time	Development phases fall behind schedule	Milestones missed	High	Allocate dedicated weekly project time and track progress against the Gantt chart	Reduce scope of current phase and carry remaining work into next phase	Review Gantt chart progress weekly
Team member illness or personal circumstances	Personal Scheduling	Unforeseen personal circumstances	One member unable to contribute for an extended period	Project falls behind schedule	Low	Maintain shared documentation so either member can continue the other's work if necessary	Redistribute immediate tasks to available member and adjust timeline	Check in with each other weekly
Insufficient participants recruited	User Recruitment	Low response rate from first-year Python students	Evaluation conducted with too few participants for meaningful results	Evaluation findings lack statistical validity	Medium	Begin recruitment early with supervisor assistance and target at least 15 participants per evaluation	Begin recruitment early with supervisor assistance and target at least 30 participants per evaluation	Monitor recruitment numbers four weeks before evaluation begins
Firebase free tier limits exceeded	Resource	Higher than expected database read and write volume during evaluation	Firebase blocks requests beyond free tier limits	System becomes inaccessible during evaluation	Low	Monitor Firebase usage throughout development and optimise database queries	Upgrade to Firebase paid tier if free tier limits are approached	Review Firebase usage dashboard weekly during evaluation
Ethics clearance taking longer than expected	Timeline	Faculty of Science Research Ethics Committee processing delays	User testing cannot begin until clearance is obtained	Evaluation phase delayed	Medium	Submit ethics application immediately after proposal is approved by 11 May 2026 deadline	Conduct additional functional and performance testing while awaiting clearance	Track ethics application status weekly
Pre and post test results inconclusive	Evaluation	Insufficient learning gain within the evaluation period	No statistically measurable improvement detected	Primary success criterion not met	Medium	Design pre and post tests carefully to detect small but meaningful learning gains	Report findings honestly and discuss limitations in the project paper	Review test design with supervisor before evaluation begins

Table A1: outlines the key risks identified for this project, together with their causes, triggers, impacts, probabilities, mitigation strategies, contingency plans, and monitoring approaches

APPENDIX B: GANTT CHART

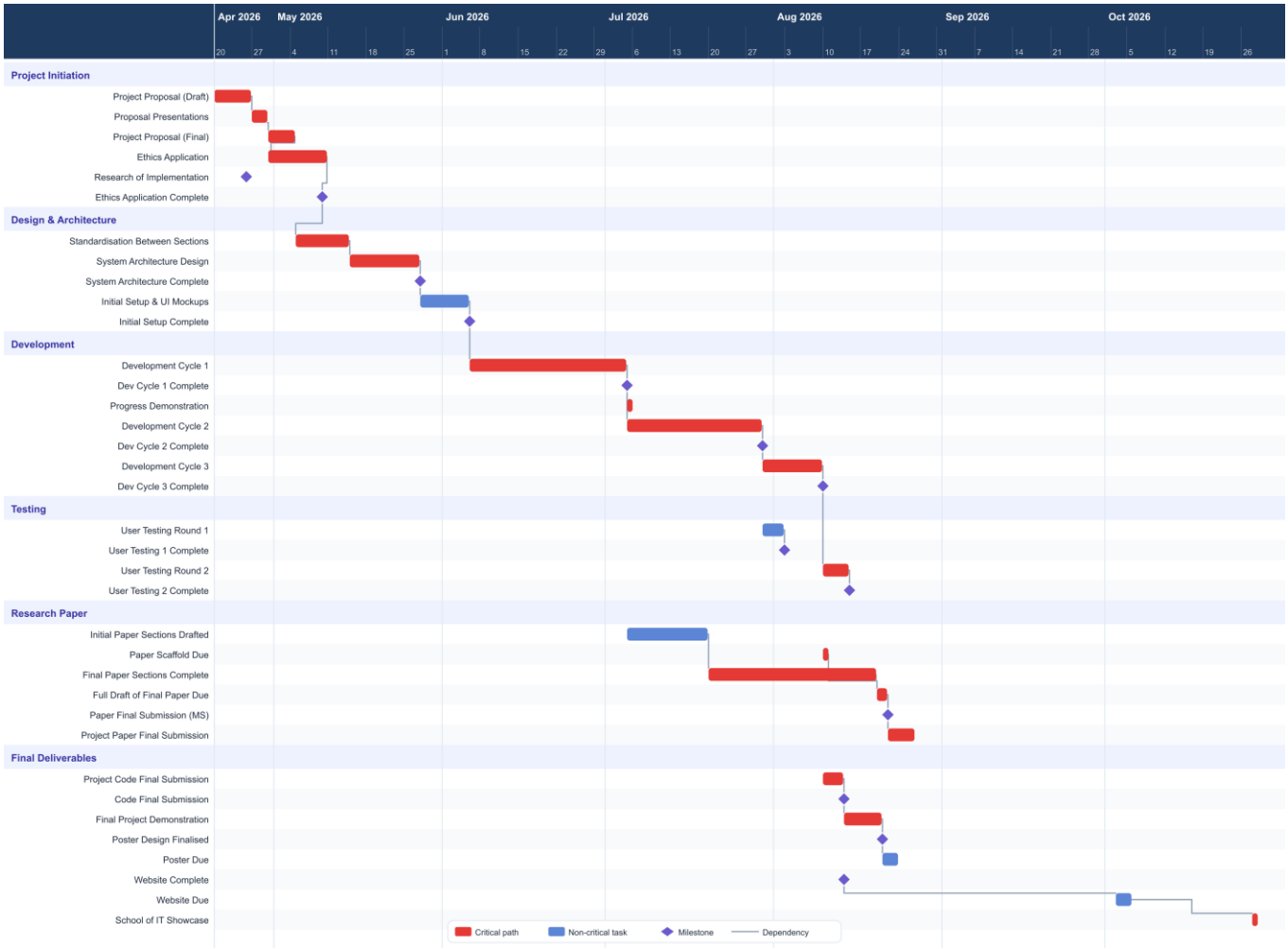


Figure B1: illustrates the project schedule, including development phases, integration, and testing milestones across the academic year.

APPENDIX C: PROJECT DELIVERABLES & MILESTONES

Deliverables

Deliverable	Due Date
Project Proposal (Draft)	26 April 2026
Project Proposal (Presentations)	28 – 30 April 2026
Project Proposal (Final)	05 May 2026
Ethics Application Deadline	11 May 2026
Project Progress Demonstration	27 July 2026
Project Paper Scaffold Due	10 August 2026
Full Draft of Final Paper Due	24 August 2026
Project Paper Final Submission	04 September 2026
Project Code Final Submission	14 September 2026
Final Project Demonstration/Presentation	16 – 23 September 2026
Poster Due	28 September 2026
Website Due	05 October 2026
School of IT Showcase	28 October 2026

Table C1: lists the concrete outputs the project will produce with their associated due dates across the academic calendar.

Milestones

Milestone	Target Date
Research of implementation complete	3 May
Ethic application complete	3 May
Standardisation Between Sections complete	15 May
System architecture complete	28 May
Initial setup of the entire system and UI mockups	6 June
Development cycle 1 complete: with continuous feedback	5 July
Initial Sections of Research Paper Drafted	20 July
Development cycle 2 complete	30 July
User testing 1 complete	3 August
Development cycle 3	10 August
User Testing 2	15 August
Final Sections Research Paper Complete	20 August 2026
Draft of Full Research Paper Complete	23 August 2026
Project Paper Final Submission	30 August 2026
Final code Submission	10 September 2026
Poster Design Finalized	25 September 2026
Website complete	02 October 2026

Table C2: lists the key technical achievements and checkpoints for evaluating progress toward project completion

APPENDIX D: SYSTEM ARCHITECTURE DIAGRAMS

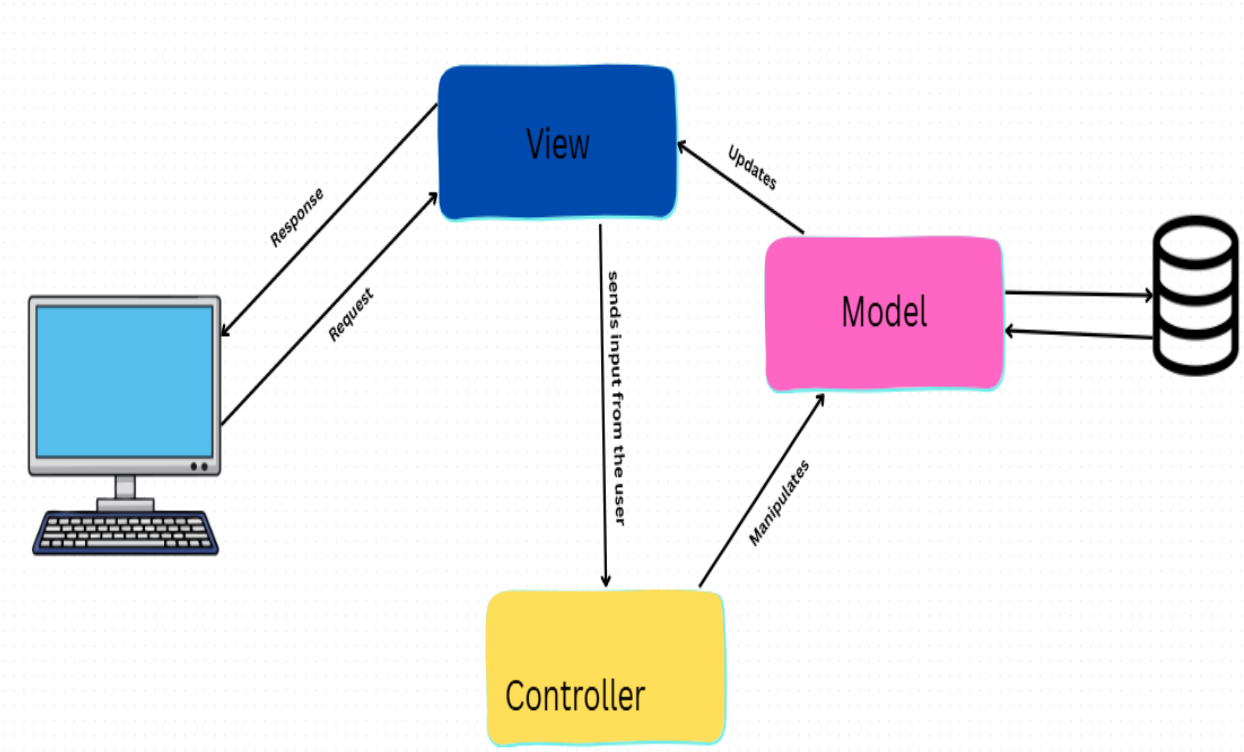


Figure 1: MVC Architecture Overview

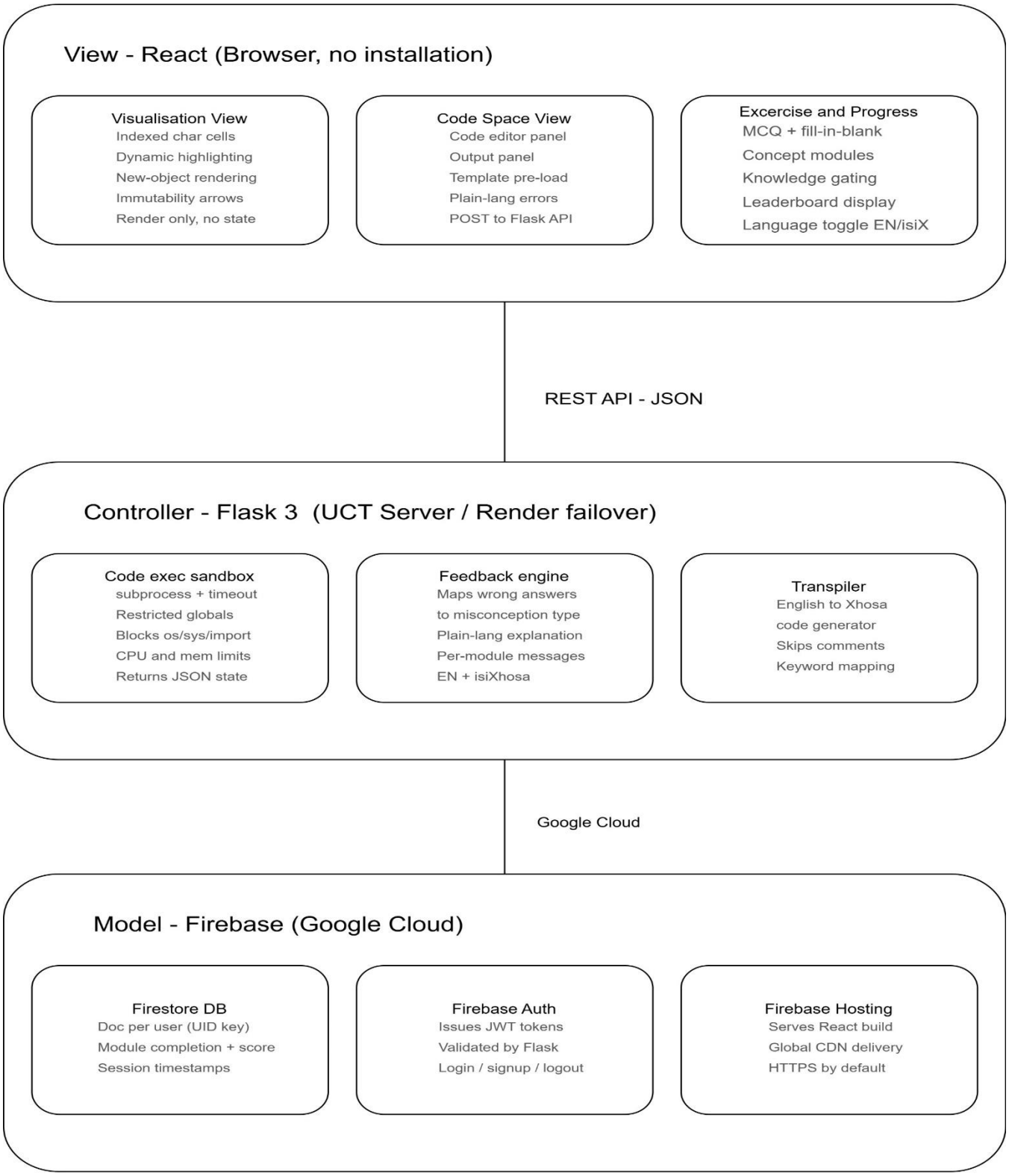


Figure 2: Detailed MVC Component Interactions